

Chapter 74

Machine Learning

Question Bank

Answer the ML questions that come up across screening calls, live-coding rounds, and case interviews for Data Scientist, ML Engineer, and analytics-adjacent roles · explain bias-variance, not just define it · spot data leakage before a model's score fools you

Approved chapter · v1

Contents

Chapter 74. Machine Learning Question Bank

- 74.1 Bias, variance, and the shape of a good model
- 74.2 Basic-but-tricky ML questions
- 74.3 Metrics, calibration, and thresholds
- 74.4 Data leakage
- 74.5 Feature engineering
- 74.6 Model debugging
- 74.7 ML case studies and live-coding walk-throughs
- Common mistakes and how to spot them
- In the real world: the feature importance that gave away the bug
- Tools
- The project
- Final-week revision list
- Key terms
- Where this leads

Chapter 74. Machine Learning Question Bank

Part VIII — The Interview Playbook

You will learn to: answer the ML questions that come up across screening calls, live-coding rounds, and case interviews for Data Scientist, ML Engineer, and analytics-adjacent roles · explain bias-variance, not just define it · spot data leakage before a model's score fools you · read a confusion matrix, an ROC curve, and a calibration plot the way an interviewer actually wants · debug a model that "worked in training and broke in production."

How this chapter is built. Same format as Chapters 70–73: every core question leads with a "Remember it as..." hook, a one-line answer, a compact tier table. Rapid-fire sections are scan tables. **This chapter draws directly on Part IV of this same book** (Chapters 35–43), which this author wrote earlier and has exact, verified section numbers and real, already-checked results for, not "chapter-level, to be confirmed" pointers like several other banks in this part. Every "Learn it in" reference below is precise. New claims specific to this chapter were freshly computed and verified the same way.

A note on scope. This is an *interview* bank, testing whether you can explain and reason about ML concepts under time pressure, not a re-teaching of the algorithms themselves (that's Chapters 35–43 in full). Read this chapter after those, not instead of them.

74.1 Bias, variance, and the shape of a good model

Q74-001 · Explain bias-variance trade-off, and prove it on a real model

Remember it as: Too simple, and the model can't learn the pattern even from the training data (high bias). Too complex, and it memorizes the training data's noise instead of the real pattern (high variance).

Answer in one line: **Bias** is error from a model too simple to capture the real pattern, showing up as poor performance on *both* training and test data; **variance** is error from a model too sensitive to the specific training sample, showing up as a large gap between strong training performance and weak test performance.

Verified, live: the same dataset, decision trees at increasing depth:

```
for d in [1, 2, 3, 4, 5, 8, None]:
    tree = DecisionTreeClassifier(max_depth=d, random_state=74).fit(X_train, y_train)
    # train acc, test acc
```

```
depth=1 (too shallow): train 0.651  test 0.600  -- both low, close together: high bias
depth=5 (near the sweet spot): train 0.894  test 0.700  -- the best test score in this sweep
depth=None (unconstrained): train 1.000  test 0.653  -- perfect on training, worse on test: high variance
```

Tier	What to say
Passes	Defines both terms correctly, can't demonstrate the trade-off with numbers
Strong	The real sweep above: depth 1 underfits (both scores low and close), unconstrained depth overfits (perfect training, a large gap to test), depth 5 sits closest to the sweet spot on this data
Extra points	+ [Validate] the actual measured numbers above, not a textbook illustration + [Business] this exact trade-off is why Chapter 37's tuned gradient boosting (test AUC 0.839) beat both an unconstrained single tree and a heavily regularized one on Riverstone's lead data, and why "just add more model complexity" is not a free lunch

Likely follow-ups: How would you diagnose which one your model has, without a controlled sweep like this? What's the relationship between bias-variance and a learning curve? **Red flag:** defining the terms correctly but being unable to say which symptom (both scores low vs. a train-test gap) points to which problem. **Learn it in:** Chapter 37, §37.10 (bias-variance with learning curves).

Q74-002 · Why can a model reach perfect training accuracy on pure noise, with enough features and too few samples?

Remember it as: Give a model enough dimensions relative to how much data it has, and it can always find some combination that happens to separate the training rows perfectly, real pattern or not.

Answer in one line: With enough features relative to the sample size, a flexible model can find a combination of features that perfectly separates the training labels by chance alone, even when the features carry zero real signal: the **curse of dimensionality** meeting overfitting.

Verified, live: 200 samples, purely random noise features, a purely random target with no relationship to any feature:

```
for n_features in [5, 50, 200]:
    X = rng.normal(0, 1, (200, n_features)) # pure noise
    # y is independent random 0/1, no real signal anywhere
```

```
5 features:  train AUC 0.670  test AUC 0.486
50 features: train AUC 0.849  test AUC 0.437
200 features: train AUC 1.000  test AUC 0.575
```

With 200 noise features and 200 samples, the model reaches a **perfect 1.000 training AUC** on data that has genuinely no signal at all, while its test AUC stays right around the 0.5 you'd expect from random guessing.

Tier	What to say
Passes	"More features can cause overfitting" (true, no demonstration of the mechanism or magnitude)
Strong	Explains the mechanism: with enough dimensions, there's almost always <i>some</i> direction in feature space that happens to align with the training labels by pure chance

Tier	What to say
Extra points	+ [Validate] the real, striking numbers above: perfect training performance on data engineered to have zero true signal + [Business] this is exactly why a model with hundreds of engineered features and only a few thousand rows needs real regularization and real held-out validation, not just a high training score, to be trusted at all

Likely follow-ups: How does regularization (L1/L2) address this specifically? What sample-to-feature ratio would make you nervous? **Red flag:** treating a high training score alone as evidence a model is good. **Learn it in:** Chapter 37, §37.2 (ridge/lasso/elastic net) and §37.10.

Rapid-fire, 74.1

#	Question	One-line answer	Extra point
Q74-003	What's regularization, in one sentence?	A penalty added to the loss function that discourages overly large or overly complex model parameters, trading a little training fit for better generalization	[Learn it in] Chapter 37, §37.2, where ridge, lasso, and elastic net are each worked by hand and compared
Q74-004	L1 vs. L2 regularization, what's the practical difference?	L1 (lasso) can shrink coefficients exactly to zero, performing feature selection; L2 (ridge) shrinks them toward zero but rarely to exactly zero	[Learn it in] Chapter 37, §37.2's lasso path figure shows this directly
Q74-005	What's a learning curve, and what does a persistent gap between train and validation lines tell you?	A plot of training and validation score against training-set size; a gap that doesn't close as more data is added signals high variance that more data alone may not fully fix	[Learn it in] Chapter 37, §37.10
Q74-006	Does adding more training data fix high bias?	No: high bias means the model itself is too simple for the pattern; more data of the same kind won't change that, a more expressive model or better features will	[Edge cases] more data reliably helps high <i>variance</i> , not high <i>bias</i>

74.2 Basic-but-tricky ML questions

Q74-007 · Is a model with 95% accuracy always a good model?

Remember it as: Always ask the base rate before believing an accuracy number. Predicting "no" for everyone on a 5%-positive problem is already 95% accurate.

Answer in one line: No: on an imbalanced problem, a model that predicts the majority class for every single row can achieve very high accuracy while catching zero of the minority class it was actually built to find.

Verified, live, using Chapter 37's own Riverstone churn data (9.7% churn rate):

```
predicting "no churn" for every account: accuracy = 90.3%
```

Tier	What to say
Passes	"Accuracy isn't everything" (correct, no concrete demonstration)
Strong	Computes or cites the actual base-rate accuracy, showing how little a headline accuracy number proves on its own
Extra points	+ [Business] this exact number, 90.3%, is the real base rate this book's own churn model had to beat, and it's why Chapter 39 insists on precision, recall, and PR-AUC instead of accuracy on any imbalanced problem

Likely follow-ups: What metrics would you ask for instead? How imbalanced does a problem need to be before accuracy becomes actively misleading? **Red flag:** treating a single accuracy number as sufficient evidence of a good model with no mention of the base rate. **Learn it in:** Chapter 39, §39.1–39.2.

Q74-008 · Can a model have a good ROC-AUC and still be a bad choice for the business?

Remember it as: ROC-AUC measures whether a model ranks well. It says nothing about whether the threshold you'll actually use makes money.

Answer in one line: Yes: ROC-AUC measures a model's ability to rank positive cases above negative ones across every possible threshold, but says nothing about calibration, about performance at the one threshold you'll actually deploy, or about whether the business cost of a false positive versus a false negative makes that threshold worthwhile at all.

Tier	What to say
Passes	"AUC only tells you ranking, not the actual threshold" (correct, no elaboration)
Strong	Names the specific gaps: calibration (Q74-014), threshold-specific cost trade-offs, and PR-AUC's sensitivity to base rate that plain ROC-AUC hides
Extra points	+ [Business] this book's own lead-scoring model has an ROC-AUC of 0.823, which sounds strong, and its real business value only became visible once a cost-based threshold was applied on top: the difference between calling every lead (₹1.08M profit) and calling the right 627 leads (₹2.45M profit)

Likely follow-ups: What would you look at alongside AUC to judge business readiness? When is a lower-AUC but better-calibrated model the right choice? **Red flag:** treating AUC as a complete summary of "is this model good." **Learn it in:** Chapter 39, §39.2 and §39.5.

Rapid-fire, 74.2

#	Question	One-line answer	Extra point
Q74-009	Does a higher R^2 always mean a better regression model?	No: R^2 mechanically increases (or stays the same) every time you add a feature, even a useless one; adjusted R^2 or held-out performance is needed to judge whether a feature genuinely helps	[Learn it in] Chapter 37, §37.1
Q74-010	Is a model with zero training error a good sign?	Usually a red flag, not a good one: it often means the model has memorized the training set rather than learned a generalizable pattern	[Learn it in] Chapter 37, §37.6 (tree depth overfit: train 1.000 / valid 0.557)

#	Question	One-line answer	Extra point
Q74-011	Do you always need to scale features before training a model?	Depends on the algorithm: distance-based methods (k-NN, k-means, SVM, regularized linear models) need it; tree-based methods (random forest, gradient boosting) generally don't	[Learn it in] Chapter 37, §37.4 (k-NN's scaling collapse: 0.765 → 0.633 unscaled)
Q74-012	Is a random forest always better than a single decision tree?	No, but it's very often more stable: a single tree is high-variance and sensitive to its exact training sample; averaging many trees reduces that variance, generally at some cost in interpretability	[Learn it in] Chapter 37, §37.7

74.3 Metrics, calibration, and thresholds

Q74-013 · Build a confusion matrix by hand and derive precision, recall, and F1 from it

Remember it as: Precision reads across the "predicted positive" row. Recall reads down the "actually positive" column. They answer different questions and usually trade against each other.

Answer in one line: Given TP, FP, FN, TN: **precision** = $TP/(TP+FP)$ (of what you flagged, how much was right), **recall** = $TP/(TP+FN)$ (of what was actually positive, how much did you catch), **F1** = the harmonic mean of the two.

Verified, live, from this book's own lead-scoring model at a 0.1 threshold:

```
TP=100  FP=386  FN=46  TN=1693
precision = 100/486 = 0.206
recall    = 100/146 = 0.685
F1        = 2 × 0.206 × 0.685 / (0.206 + 0.685) = 0.317
```

Tier	What to say
Passes	States the formulas correctly, makes an arithmetic error applying them
Strong	The correct worked calculation above, matching a real model's real confusion matrix
Extra points	+ [Business] at this specific threshold, only 1 in 5 flagged leads actually converts (20.6% precision), but the model catches over two-thirds of all real conversions (68.5% recall): whether that trade-off is right depends entirely on what a false positive costs versus what a missed lead costs, exactly Chapter 39's cost-based threshold logic

Likely follow-ups: What happens to precision and recall as you raise the threshold? Why can't you maximize both at once, in general? **Red flag:** computing precision and recall with the numerator and denominator swapped, a very common slip under time pressure. **Learn it in:** Chapter 39, §39.1.

Q74-014 · What does it mean for a model's probabilities to be "well calibrated," and why can a model with good ranking still be poorly calibrated?

Remember it as: Ranking asks "did the model put the right cases higher than the wrong ones?" Calibration asks "when it says 70%, does that group of cases actually convert 70% of the time?" Different questions, no guaranteed relationship.

Answer in one line: A model is well calibrated if, among all the cases it assigns a given probability (say, 70%), roughly that share actually turn out positive; a model can rank cases perfectly (excellent AUC) while still being badly calibrated, if it's simply confident in the wrong direction, systematically too high or too low.

Verified, live, from this book's own comparison of two models on identical data:

Naive Bayes:	mean predicted probability 18%	actual positive rate 6.6%	ROC-AUC 0.806
top decile:	predicts 94%	actual rate in that decile 27%	

Naive Bayes ranks reasonably well (0.806 AUC) while being **dramatically overconfident**: its top decile predicts a 94% chance of a positive outcome for cases that are actually positive only 27% of the time.

Tier	What to say
Passes	"Calibration means the probabilities are accurate" (vague, no example of the ranking/calibration gap)
Strong	The real example above: a model can have good AUC and still be badly, provably miscalibrated
Extra points	+ [Business] Naive Bayes's overconfidence here isn't a hypothetical concern; a rep told "94% chance" who wins one in four stops trusting the model entirely, which is why isotonic calibration (fixing log loss from 0.483 to 0.202 with almost no change to ranking) matters as much as the ranking score itself

Likely follow-ups: How do you check calibration in practice? What's the difference between Platt scaling and isotonic regression? **Red flag:** assuming a high AUC implies trustworthy probabilities. **Learn it in:** Chapter 39, §39.4.

Rapid-fire, 74.3

#	Question	One-line answer	Extra point
Q74-015	ROC-AUC vs. PR-AUC, when does the choice matter most?	They agree closely on balanced data; on imbalanced data, PR-AUC reflects what users actually experience (precision) far better than ROC-AUC, which stays deceptively high	[Learn it in] Chapter 39, §39.2 (0.823 ROC-AUC vs. 0.302 PR-AUC, same model, same data)
Q74-016	What's the harm in always using 0.5 as a classification threshold?	It's an arbitrary default with no connection to the actual cost of a false positive vs. a false negative; on Riverstone's lead data it flags only 7 of 2,225 leads, worse than predicting "no" for everyone	[Learn it in] Chapter 39, §39.1 and §39.5
Q74-017	What's MAE vs. RMSE, and when do they disagree the most?	Mean absolute error weights every error equally; root mean squared error punishes large errors disproportionately (squaring), so they diverge most when a few large outlier errors exist	[Learn it in] Chapter 39, §39.3 (MAE ₹92,933 vs. RMSE ₹232,567, same model)
Q74-018	What's the difference between macro and weighted F1 for a multi-class problem?	Macro averages each class's F1 equally regardless of class size; weighted averages proportionally to each class's frequency, so a poor score on a rare class matters less in the weighted version	[Business] macro F1 is usually the fairer choice when a rare class is exactly the one you care most about catching

74.4 Data leakage

Q74-019 · Demonstrate target leakage: build a model with and without a leaked feature, and show the difference

Remember it as: *If a feature couldn't have existed at the moment you're predicting, it doesn't belong in the model, however good it makes the score look.*

Answer in one line: **Target leakage** happens when a feature is influenced by, or nearly a copy of, the outcome you're trying to predict, giving the model information it would never actually have at prediction time, and inflating its score in a way that will not hold up in production.

Verified, live:

```
leaked_feature = y + rng.normal(0, 0.1, n)    # e.g. a status field only ever set AFTER the outcome
is known

without leaked_feature: test AUC 0.924
with leaked_feature:    test AUC 1.000
feature importances with leak: signal 0.306, leaked_feature 0.694
```

A feature that's simply a noisy copy of the answer pushes test AUC to a **suspicious, essentially perfect 1.000**, and dominates feature importance at nearly 70%, exactly the profile of a leak, not a genuinely powerful predictor.

Tier	What to say
Passes	"Leakage is when future information gets into training" (correct, no demonstration)
Strong	The worked comparison above, and the diagnostic instinct it teaches: a suspiciously high score (especially near-perfect) plus one feature dominating importance is the classic leakage fingerprint
Extra points	+ [Validate] the real jump from 0.924 to 1.000, and the real importance split, not an assumed pattern + [Business] this exact mechanism, a single leaked column inflating an AUC from 0.823 to 0.978, is what Chapter 36's own lead-scoring case study found and removed before shipping the model

Likely follow-ups: Name three real-world sources of target leakage beyond a literal copy of the label. How would you catch this without already suspecting a specific column? **Red flag:** treating a near-perfect score as good news rather than the first thing to investigate. **Learn it in:** Chapter 36, §36.5 (four separate leak types demonstrated, one at a time, with exact AUC inflation for each).

Q74-020 · What's temporal leakage, and why is a random train/test split dangerous for time-ordered data?

Remember it as: *A model trained on the future to predict the past will look brilliant and mean nothing. Always split by time when time is part of the story.*

Answer in one line: **Temporal leakage** happens when information from *after* the prediction moment sneaks into training, most commonly via a random (not time-based) train/test split on time-series or event data, letting the model implicitly learn from outcomes that hadn't happened yet at the point it's meant to predict.

Tier	What to say
Passes	"You should split time series data by time, not randomly" (correct, no reasoning given)
Strong	Explains that a random split lets rows from <i>before</i> a test-set row's date sit in the training set alongside rows from <i>after</i> it, letting features that quietly encode "what eventually happened" leak through
Extra points	+ [Business] this book's own lead model scored <i>lower</i> on a random split (0.791) than a proper time split (0.823), the <i>opposite</i> of what leakage usually does, and the honest explanation matters: it wasn't a leakage problem, it was a genuine shift in lead mix over time that the random split happened to average away: the lesson being that a lower score isn't automatically "more honest," you have to know <i>why</i> the numbers differ

Likely follow-ups: How would you build a time-respecting cross-validation scheme, not just a single split? What's a rolling-origin backtest? **Red flag:** assuming every score difference between a random and time-based split is automatically explained by leakage. **Learn it in:** Chapter 36, §36.3 and §36.9; Chapter 40, §40.7 (backtesting).

Rapid-fire, 74.4

#	Question	One-line answer	Extra point
Q74-021	What's target encoding leakage?	Encoding a categorical feature using the target's own mean, computed on the <i>same</i> rows being predicted, silently baking outcome information into the feature	[Learn it in] Chapter 36, §36.5 (target encoding contamination: 0.845 vs. 0.823, the honest version)
Q74-022	Why is fitting a scaler or encoder on the full dataset before splitting a leakage risk?	The scaler's parameters (mean, std, category list) are computed using test-set information, technically letting a little bit of the test set influence training	[Learn it in] Chapter 36, §36.8 (pipelines: encode-first CV 0.82 vs. proper pipeline 0.78)
Q74-023	What's the fastest sanity check for suspected leakage in a model that scores too well?	Check what happens if you remove features one at a time, especially any that are suspiciously predictive alone, or any recorded at or after the outcome moment	[Business] a single feature responsible for a large share of feature importance, especially one that "sounds like" the answer, is the single most common leakage fingerprint
Q74-024	Can leakage happen even with a properly time-ordered split?	Yes: a feature aggregated "as of today" that actually includes same-day or future data (an off-by-one in a rolling window, say) leaks even inside a correctly time-split pipeline	[Learn it in] Chapter 40, §40.6 (a leaky rolling-mean feature, and why the score barely changed: leaks aren't always dramatic)

74.5 Feature engineering

Q74-025 · A categorical feature has 500 unique values. What are your options, and what's the trade-off for each?

Remember it as: One-hot blows up your feature count. Target encoding risks leakage. Frequency or hashing keeps it small and safe, at the cost of some information.

Answer in one line: One-hot encoding (creates 500 new columns, sparse and potentially unwieldy for a high-cardinality feature), target encoding (compact, but a real leakage risk unless done with proper out-of-fold encoding, Q74-021), frequency encoding (replace each category with how often it appears, simple and safe, loses category-specific signal), or feature hashing (a fixed-size compressed representation, at the cost of occasional hash collisions between unrelated categories).

Tier	What to say
Passes	Names one-hot encoding as the only option, without acknowledging it scales poorly with cardinality
Strong	Names at least two alternatives with the trade-off for each, above
Extra points	+ [Business] the right choice depends on the model: tree-based models tolerate high-cardinality categoricals reasonably well without heavy encoding; linear and distance-based models generally need the dimensionality actively managed

Likely follow-ups: How would you safely do target encoding without leaking? What's the risk of a rare category appearing in test data but not training data? **Red flag:** treating one-hot encoding as the universal default regardless of cardinality. **Learn it in:** Chapter 36, §36.6.

Rapid-fire, 74.5

#	Question	One-line answer	Extra point
Q74-026	Why log-transform a skewed numeric feature like revenue before modeling?	Compresses a long right tail, reduces the influence of extreme outliers on linear-model coefficients and distance-based methods, and often makes the feature's relationship with the target closer to linear	[Learn it in] Chapter 36, §36.6
Q74-027	What's an interaction feature, and when do you need one explicitly?	A feature built from combining two others (a product, a ratio, a difference) to capture an effect that depends on both together; needed explicitly for models (like linear regression) that don't discover interactions automatically the way trees can	[Learn it in] Chapter 36, §36.6
Q74-028	Should missing values always be imputed with the mean?	No: sometimes the fact that a value is <i>missing</i> is itself informative (worth a separate missing-indicator flag), and imputing blindly can hide that signal	[Learn it in] Chapter 36, §36.7 (blanks winning 5.71% vs. 8.06%, a real case where missingness itself mattered)
Q74-029	What's the risk of engineering features using information "as of today" instead of "as of the prediction moment"?	It silently recreates temporal leakage (Q74-020): a feature aggregated using data that wouldn't have existed yet at the real prediction time	[Learn it in] Chapter 36, §36.6's "aggregates as of prediction moment" discipline

74.6 Model debugging

Q74-030 · A model scores well in offline evaluation but performs noticeably worse once deployed. Walk through how you'd debug it

Remember it as: "It worked in training" and "it works in production" are different claims. The gap is almost always a difference between what the model saw then and what it's seeing now.

Answer in one line: Check, in order: **training-serving skew** (are production features computed the same way as training features, with the exact same logic and timing), **data drift** (has the real-world distribution of inputs shifted since training), **label delay or leakage removed** (was a feature available in training that's genuinely not available at real serving time), and **silent pipeline bugs** (a join, a default value, a type conversion behaving differently in production).

Tier	What to say
Passes	"Maybe the data changed" (a reasonable first guess, no structured process)
Strong	The four-category checklist above, with a proposed order to check them in
Extra points	+ [Real evidence] this book's own churn model's real feature importance ranking (days since last order, product breadth, late payments) gives a template for what to check first: recompute those specific features in production and compare their distributions directly against training + [Business] the fastest, cheapest check is almost always comparing a production feature's distribution against its training-time distribution for the same population; a large shift there usually explains most of the gap before touching the model itself

Likely follow-ups: How would you set up ongoing monitoring to catch this before a stakeholder notices? What's the difference between concept drift and data drift? **Red flag:** jumping to "retrain the model" as the first response, before diagnosing what actually changed. **Learn it in:** Chapter 39, §39.9 (model cards, monitoring) and Chapter 52 (deployment and monitoring, referenced ahead).

Rapid-fire, 74.6

#	Question	One-line answer	Extra point
Q74-031	Your model's feature importances put a customer ID or row-order column near the top. What does that suggest?	A strong signal of leakage or a data-pipeline bug (an unshuffled time-based split, an ID that correlates with signup cohort, or similar), not a real predictive relationship	[Business] an ID column should never be genuinely predictive on its own; if it looks like it is, something else is wrong upstream
Q74-032	Training loss keeps falling but validation loss starts rising partway through training. What's happening, and what do you do?	Overfitting in progress; use early stopping (stop training at the epoch with the best validation score) rather than training to the lowest possible training loss	[Learn it in] Chapter 40, §40.5's Holt-Winters overfitting to in-sample error is the same underlying idea in a non-neural-network setting
Q74-033	A model performs very differently across two customer segments. Is that automatically a problem?	Not automatically, but it needs investigation: check whether it reflects a genuine difference in the segments' base rates (calibrated correctly for both) or an unfair, uncalibrated gap	[Learn it in] Chapter 39, §39.8 (fairness checks by segment)

#	Question	One-line answer	Extra point
Q74-034	You retrain a model monthly and its performance quietly degrades over several retraining cycles. What would you check first?	Whether the training data itself has drifted (a changing lead mix, a new customer segment, a process change upstream) before assuming the modeling approach itself has stopped working	[Learn it in] Chapter 36, \$36.3 (marketplace share rising from 30% to 44.5% over the same period this book's data covers)

74.7 ML case studies and live-coding walk-throughs

Q74-035 · Case study: a vendor pitches a "91% accurate" churn model. Walk through how you'd evaluate the claim, live

What they're really testing: whether you reach for the base rate and the right metrics before being impressed by a headline number, under real conversational pressure.

Talked through live: "First question: what's the actual churn rate in their data? If it's anywhere near 9%, predicting 'no churn' for everyone would already be about 91% accurate, so that number alone tells me almost nothing yet."

Verified, live, using this book's own real numbers:

Riverstone's actual churn rate: 9.7%
predicting "no churn" for everyone: 90.3% accuracy

Talked through, live: "Their 91% is barely above the do-nothing baseline of 90.3%. I'd ask for AUC and log loss compared against that same baseline, not accuracy, and I'd ask what their model was actually trained on, since a deep-learning label alone tells me nothing about whether it beats a well-tuned simpler model."

Extra-points moves demonstrated: **[Clarify]** asked for the base rate before reacting to the headline number. **[Validate]** the real comparison, not an assumed one. **[Business]** proposed the concrete next step (ask for AUC/log loss against the baseline) rather than just expressing skepticism.

Likely follow-ups: What if the vendor's AUC actually did beat your own model's? What questions would you ask about their training data before trusting it? **Red flag:** being impressed by a percentage with no base-rate context. **Learn it in:** Chapter 43's own real-world story (a near-identical vendor pitch, with the base-rate check as the deciding question).

Q74-036 · Case study: which algorithm would you reach for first, and how would you know if you're wrong?

What they're really testing: whether "it depends" gets followed by an actual decision process, not left as a dodge.

Talked through live: "For a first pass on a new tabular business problem, I'd start with logistic regression as a fast, interpretable baseline, then move to gradient boosting as the strong default, and I'd know I picked wrong if boosting doesn't meaningfully beat logistic regression on held-out data,

since that would suggest the relationships in the data are mostly linear anyway and the added complexity isn't earning its keep."

Verified, live, from this book's own comparison of exactly this decision, on Riverstone's lead data:

```
logistic regression (test): 0.797
tuned gradient boosting (test): 0.839
```

Talked through, live: "In this specific case boosting won by a real, meaningful margin, so the added complexity was worth it. But on a different dataset in this same book, an engineered logistic regression model actually tied or beat tuned boosting, which is the whole reason to always run the baseline, not skip straight to the fancier model on faith."

Extra-points moves demonstrated: [Trade-offs] named a concrete decision rule ("did the complex model meaningfully beat the baseline") instead of a vague "try a few things." [Real evidence] cited two different, real, contradicting outcomes from the same body of work, showing the answer genuinely depends on the data rather than reciting "boosting always wins."

Likely follow-ups: How would you decide "meaningfully beat," in numbers? What would make you skip straight to a more complex model without trying logistic regression first? **Red flag:** naming one algorithm as a universal answer with no acknowledgment that it depends on the data. **Learn it in:** Chapter 37, §37.11–37.12 (the full tuning and final comparison), and Chapter 43, §43.5 (the case where a small neural network landed *between* logistic regression and boosting).

Common mistakes and how to spot them

Mistake	Symptom	Fix
Reporting accuracy alone on an imbalanced problem	A headline number barely above the base rate	Always state the base rate; use precision/recall/PR-AUC
Treating high AUC as proof of a trustworthy probability	A confidently wrong "94% chance" shown to a stakeholder	Check calibration separately from ranking (Q74-014)
A suspiciously perfect or near-perfect score	The model has learned the target from a leaked feature	Check feature importance for one dominant column; question the top feature specifically
A random train/test split on time-ordered data	An offline score that doesn't hold up once deployed	Split by time; backtest with rolling origins
Jumping to "retrain the model" when production performance drops	The real cause (a pipeline bug, drifted inputs) goes unaddressed	Diagnose training-serving skew and data drift before retraining
Assuming more features always helps	Perfect training score, poor test score, especially with few samples	Check sample-to-feature ratio; regularize; validate on held-out data
Picking a complex model without a baseline	No way to tell whether the complexity was worth it	Always run the simplest reasonable baseline first, and compare

In the real world: the feature importance that gave away the bug

Priyanka, a Data Scientist candidate, is shown a model with a suspiciously strong 0.97 AUC and asked to find what's wrong. Rather than guessing, she asks to see the feature importance table first, and immediately flags the top feature: `days_since_status_change`, contributing nearly half the model's total importance.

She reasons out loud: "That column name suggests it's updated *when* the status changes, which for a churn model probably means it gets updated right around, or even after, the churn event itself. If that's true, the model isn't predicting churn, it's detecting that churn has already been recorded." The interviewer confirms: the feature was indeed set at the moment an account was marked churned, a close cousin of this chapter's own leaked-feature demonstration.

The interviewer's note afterward: *"Went straight to feature importance, and reasoned about the specific column name rather than asking to see more code. Diagnosed the leak from the name and the number alone, before we even opened the data."* That's this chapter's whole method in miniature: a suspicious score plus a dominant feature is the fingerprint, and naming *why* a specific feature is suspicious, not just that leakage is possible in general, is what separates a strong answer from a correct-sounding one.

Tools

scikit-learn, **pandas**, and **NumPy**, the same stack used throughout Chapters 35–43; every number in this chapter was computed with them, not recalled from memory. No new tools beyond what those chapters already introduced.

The project

Goal: reproduce this chapter's core demonstrations on your own model or data.

1. Run your own bias-variance sweep (varying max depth, regularization strength, or a similar complexity knob) on a real model, and identify where the sweet spot actually sits, not where you'd guess it sits.
 2. Deliberately leak a feature into a model you control (a noisy copy of the target, or a feature computed after the outcome) and measure exactly how much the score inflates.
 3. Take a model you've built with a random split and rebuild it with a time-based split if your data has any temporal structure; compare the two honestly.
 4. Pick one of your own models and write a two-sentence "how I'd debug this in production" plan before you need it for real.
-

Final-week revision list

Q74-001, Q74-002, Q74-007, Q74-008, Q74-013, Q74-014, Q74-015, Q74-019, Q74-020, Q74-021, Q74-025, Q74-030, Q74-031, Q74-035, Q74-036.

Key terms

bias-variance trade-off · underfitting · overfitting · regularization (L1/L2) · learning curve · curse of dimensionality · base rate · confusion matrix · precision · recall · F1 score · ROC-AUC · PR-AUC · calibration · target leakage · temporal leakage · target encoding leakage · training-serving skew · data drift · concept drift · feature importance · one-hot encoding · frequency encoding · feature hashing · interaction feature · missing-value indicator

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapters 35–43 of this same book** are where every technique this bank draws on was taught in full, with the exact section numbers cited throughout; this chapter tests it, it doesn't re-teach it.
- **Chapter 77, Data Engineering & Data System Design Bank**, covers the pipeline and infrastructure side of the training-serving skew problem in §74.6 at production scale.
- **Chapter 30 and Chapter 31** (experimentation and causal inference) are the answer whenever an interview pushes past "the model predicts X" into "does changing X actually cause the outcome."